

Assignment

Advanced Machine Learning Artificial Intelligence

Lecturer: Luciano Gerber, Indranath Chatterjee

Muhammad Zahid
25925501

May 2026

1 Data Description and Pre-Processing

The Provided dataset is focused on Car Sales Adverts from **AutoTraders**, aimed at predicting vehicle price using attributes such as mileage, make, model, fuel type, body type, year of registration, and condition. The dataset contains a mixture of numerical, categorical, and binary features, requiring appropriate pre-processing for machine learning models.

Feature Types: The target variable, price, is continuous and highly right-skewed due to the presence of high-value luxury vehicles. Numerical features: include mileage and year of registration. Categorical features: fuel type, body, make, model, colour. binary Features: vehicle condition, crossover status.

Predictive Relationships: Correlation analysis show mileage as negatively correlated with price (-0.63), while year of registration has a positive correlation (0.34). Vehicle condition is a strong categorical predictor, with newer and well-maintained vehicles showing higher median prices. Fuel type shows Majority of petrol and diesel hybrids cover upper and lower price ranges with low mileage, while the Diesel and Petrol plug in hybrid cover the lower ranges.

Missing Values and Outliers: The dataset contains missing values across several features. The reg_code feature was dropped due to redundancy with the year of registration. Outliers were identified in the year of registration (values below 1903) and corrected using group-based imputation.

IQR function was created to remove extreme outliers from price and mileage as to prevent model from overfitting. This reduced our data by 8.57% removing 34,444 rows.

```
def find_outliers_iqr(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return lower_bound, upper_bound
```

Imputation Strategy: Missing values were handled using hierarchical group-based imputation. Year of registration and Mileage were filled using medians grouped by model, then make, followed by a global median fallback method for year of registration. Categorical features such as colour, fuel type, and body type were filled using mode-based imputation with hierarchical fallback strategies.

Feature Engineering: To address skewness in the target variable, a log transformation (**log1p**) was applied to price, improving distribution symmetry and model stability without losing information and discarding extreme values. This method was applied using **TransformedTargetRegressor**. New Features vehicle age, mileage per year were generated from mileage and year of registration.

```
AT_copy_1['vehicle_age'] = 2024 - AT_copy_1['year_of_registration']
AT_copy_1['mileage_per_year'] = AT_copy_1['mileage'] / (AT_copy_1['vehicle_age'] + 1)
```

Pre-Processing: All models use **scikit-learn** Pipeline, implemented with Column-transformer. This is to ensure consistency in pre-processing and preventing data leakage during the training phase. The phase includes numerical scaling (Standard Scaler), categorical encoding (Target Encoder & One-Hot Encoder), and binary(Ordinal Encoder) encoding. This step was implemented as:

```

preprocessor = ColumnTransformer(
    transformers=[('scaler', StandardScaler(), num_feat),
                  ('onehot', OneHotEncoder(sparse_output=False, handle_unknown='infrequent_if_exist',
                  drop='if_binary'), low_card_cat_feat), ('tar_enc', TargetEncoder(smoothing=10), high_card_cat_feat),
                  ('ordinal_enc', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1), binary_cat_feat)],
    remainder='passthrough', verbose_feature_names_out=False))

```

2 Part I

2.1 Automated Feature Selection

To evaluate the impact of Automated feature selection, multiple approaches were explored within a consistent pipeline framework using Linear Regression with log-transformed targets.

Model	Features	Train R^2	Test R^2	Train MAE	Test MAE
Without Feature Selection	30	0.8896	0.8887	1827.57	1829.65
With SelectKBest	10	0.8880	0.8871	1834.93	1835.94

Table 1: Feature Selection Comparison

Baseline (No Feature Selection): A baseline model was trained on pre-processed features.

```

regr_noFS = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("regr", TransformedTargetRegressor(
            regressor=GradientBoostingRegressor(n_estimators=100, max_depth=5, random_state=42),
            func=np.log1p, inverse_func=np.expm1)))

```

The evaluation of the model was done using cross-validation, which showed The model without feature selection performs slightly better giving us 30 features. Training and Test score show similar result giving us no overfitting.

```

print("Test R2:", regr_noFS.score(X_test, y_test), "Train R2:", regr_noFS.score(X_train, y_train))
print("Test MAE:", mean_absolute_error(y_test, regr_noFS.predict(X_test)),
      "Train MAE:", mean_absolute_error(y_train, regr_noFS.predict(X_train)))

```

Best-k Feature Selection: SelectKBest with (f_regression) was applied to retain the top 10 most relevant features.

```

regr_bestk = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("featsel", SelectKBest(f_regression, k=10)),
        ("regr", TransformedTargetRegressor(
            regressor=GradientBoostingRegressor(n_estimators=100, max_depth=5, random_state=42),
            func=np.log1p, inverse_func=np.expm1)))

```

This Feature selection using **SelectKBest** reduced the feature space from 30 to 10 features; However, it does not improve prediction better than without using feature selection. This suggests similar performance without the additional features.

```

print("Test MAE:", mean_absolute_error(y_test, preds_test))
print("Train MAE:", mean_absolute_error(y_train, preds_train))
print("Train R2:", regr_bestk.score(X_train, y_train))
print("Test R2:", regr_bestk.score(X_test, y_test))

```

Other Methods: Sequential Feature Selection (SFS) and Recursive Feature Elimination (RFE) were not implemented due to their high computational cost on the large dataset. If the model is not tuned properly a single step can take extensive amount of time. Given that Best-k selection provided better computational performance with significantly lower complexity, it was chosen as the preferred method.

2.2 Tree Ensembles

Two Tree Ensemble models were implemented within a pipeline framework using `create_regr_ppln` function.

```

def create_regr_ppln(est, preprocessor):
    regr_pipe = Pipeline(
        steps=[
            ("pp", preprocessor),
            ("regr", est)])
    return regr_pipe

```

This gives consistent performance, prevents leakage from Encoders like **Target Encoder**, and gives a chance for fair comparison. Both models use a moderate number of trees ($n=50$) to avoid heavy computation and overfitting.

Gradient Boosting Regressor: The GB Regressor uses a shallow tree depth (max depth=4) to avoid overfitting, a low learning rate (0.05) for stable learning, and to prevent over-correction. A subsample of (0.8); introduces randomness, prevents over-fitting, and reduces variance.

```

gbr = create_regr_ppln(TransformedTargetRegressor(
    regressor=GradientBoostingRegressor(n_estimators=50, max_depth=4, subsample=0.8,
    learning_rate=0.05, random_state=42),
    func=np.log1p, inverse_func=np.expm1), preprocessor)

```

A Test MAE of approximately 2450 with a similar training MAE, indicating model stability and minimal overfitting. The R^2 scores for both training (0.7956) and testing (0.7962) were closely aligned, suggesting consistent predictive performance. In conclusion, the specific hyper-parameters give a well balanced performance.

Random Forest Regressor: The RF Regressor uses a default ($\sqrt{p}$) for reduced correlation. Max Depth is not set to allow full depth for the tree.

```

rfr = create_regr_ppln(TransformedTargetRegressor(
    regressor=RandomForestRegressor(n_estimators=50, max_features='sqrt', random_state=42),
    func=np.log1p, inverse_func=np.expm1), preprocessor)

```

This model indicates strong memorization of training data, with clear overfitting gap. RF Regressor shows High variance model behavior.

Random Forest achieved a significantly lower training error (MAE 621) and a higher training R^2 (0.984), but the same performance is not observed in the test set (MAE 1469, R^2 0.9251). This gap indicates mild overfitting due to high model flexibility.

2.3 Ensemble of Tree Ensembles

For Ensembling Linear Regressor was Dropped for its performance. Although Test and Train (MAE: 2,863.70, MAE: 2,870.01) provide us a stable model and low overfitting, the R2 score of (0.6666) in both Test and Train means it does not perform as well compared to Random Forest and Gradient Boosting. Hence Linear regressor is not used in Ensemble.

Before Ensemble simple 3-fold cross-validation test for negative mean absolute error gives us approximately a similar MAE score same as individual model test. The standard MAE deviation for both train and test for GBR is approx (4.7 to 4.3) for test and train solidifying our initial belief. For RFR a Train score of (0.9) and Test score of (4.5) indicates mild overfitting.

```
ensemble = VotingRegressor(estimators=[('gb', gbr), ('rf', rfr)], weights=[0.5, 0.7], n_jobs=-1)
```

The Voting Regressor was constructed using an ensemble model from both trees; a weighted average of (0.7) was applied for the Random Forest Regressor and (0.5) for the Gradient Boost Regressor. Running a 3-Fold MAE & R2 cross-validation on Ensemble gives us Voting Ensemble CV MAE & R2 score of [1734.12 (+/- 6.98) & 0.8978 (+/- 0.0017)].

The ensemble achieves a strong Test R2 (0.90) and Train (0.91), outperforming the GBR and showing robust generalization compared to the slightly overfit RFR .

2.3.1 Output Inspection

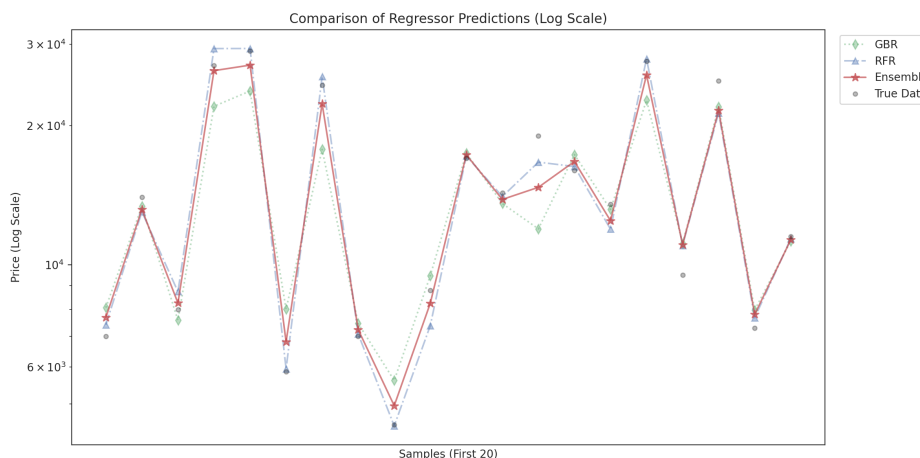


Figure 1: Comparison of Regressor Predictions (Log Scale)

The Figure 1 is a visualization of 20 samples of true data and gives us a general idea on the performance of the ensemble on a graph. **Ensemble model** is consistently closest to the true data, with GBR and RFR trailing closely. In some cases GBR seems to outperform RFR with slight under- and over-shooting. while in other cases RFR seems to trail Ensemble closely. This visually confirms Ensemble to be most stable and closest to the true data.

2.4 Feature Importance

To understand how the predictive model behaves, two interpret-ability techniques: Permutation Importance (PI) & SHAP (SHapley Additive exPlanations) are used. **Permutation Importance (PI)**: Identifies a clear cut influence of the features.

```
perm_importance_df = pd.DataFrame(dict(feature=X_test.columns,
                                       pi_mean=perm_importance['importances_mean'],
                                       pi_std=perm_importance['importances_std']))
```

This specific code block gives us PI-mean for all the features which than is sorted and plotted as shown in Figure 2.

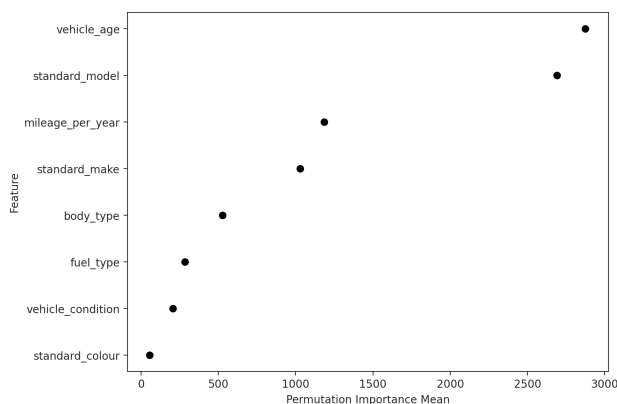


Figure 2: Permutation Importance

The model is highly influenced by by \textbf{vehicle age}, \textbf{standard model}, and \textbf{mileage per year}, with PI-Mean values of approximately 2872, 2690, and 1184 respectively. The features such as standard colour and vehicle condition show very small mostly negligible PI-Mean suggesting no real impact or inconsistency in the feature. **SHAP**

(SHapley Additive exPlanations): The SHAP summary plot for our ensemble confirms the permutation importance feature ranking. It also provides us details on the effect of the features; High values of **vehicle age**(red points) strongly decreases predicted price. Certain **models** and **manufacturers**(red points) strongly increase predicted price. Mileage per year has a consistent negative effect across all observations which confirms our previously observed belief of mileage from the heatmap. body type has a small positive effect decreasing predicted price, while fuel type similarly has a small negative impact slightly increasing the predicted price.

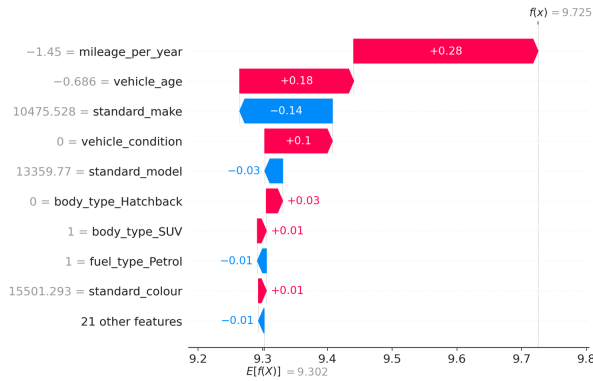


Figure 3: SHAP Waterfall Plot

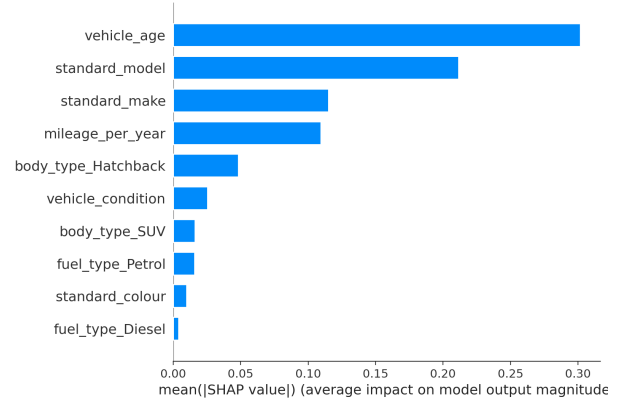


Figure 4: Mean SHAP Importance

Mean SHAP Shows similar ranking same as Permutation importance. This reinforces the stability of the model's feature dependencies.

2.5 SHAP/PDP Model Explanations

The SHAP model explanation is applied on a small portion of the ensemble model using an average voting regressor. By creating explanations for models individually we later use a Voting Regressor method for SHAP ensemble.

```
shap_sample = shap.sample(X_test_pp, 20, random_state=42) # Use a smaller sample to speed up SHAP calculation
shap_gbr = explainer_gbr(shap_sample, check_additivity=False).values # tree-path to speed up calculation
shap_rf = explainer_rf(shap_sample, check_additivity=False).values
shap_ensemble = (shap_gbr + shap_rf) / 2 # Ensemble SHAP = average; VotingRegressor averages predictions
```

The Figure 4 shows, the highest mean absolute SHAP values demonstrate the strongest average impact on predicted vehicle price. This is observed across both global and local SHAP explanations. The model exhibits behaviour consistent with real-world vehicle evaluation trends.

High values of *vehicle_age* and *mileage_per_year* consistently push predictions downward, reflecting natural depreciation and increased vehicle wear. Conversely, certain **vehicle models** and **manufacturers** contribute positively to predicted prices, capturing brand reputation and trim-level effects.

Features such as *standardcolour*, *vehicle_condition*, and *fuel_type* show comparatively small SHAP magnitudes, indicating limited predictive influence within the dataset.

Partial Dependence Plots (PDPs): were used to examine how the model's predictions change when varying a single feature while holding all other features constant.

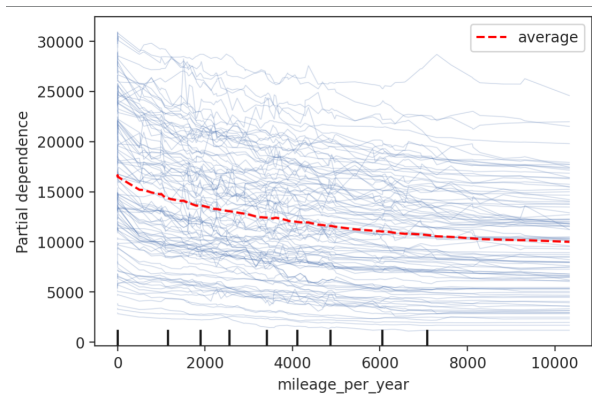


Figure 5: PDP mileage_per_year

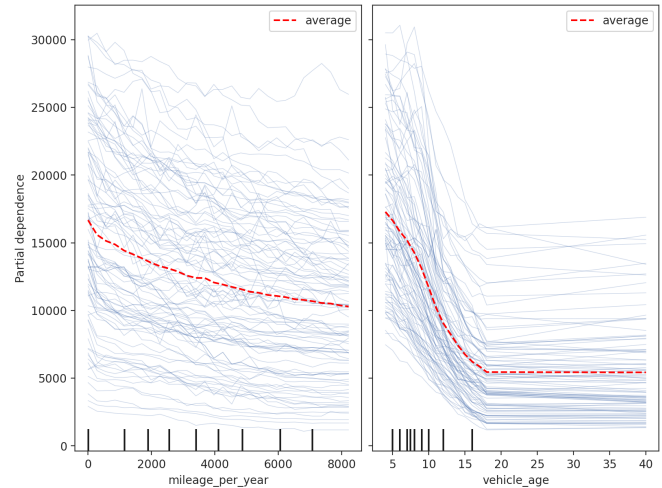


Figure 6: PDP mileage_per_year & vehicle_age

The PDP for *mileage_per_year* in Figure ?? demonstrates a clear **decreasing relationship**:

- As mileage per year increases, predicted vehicle price decreases.
- The average PDP curve declines smoothly, while individual curves show moderate variation but follow the same overall trend.

The Figure 6 indicates that the model successfully learns a realistic depreciation pattern associated with vehicle usage intensity. The PDP for *vehicle_age* shows:

- Steep decline predicted price for newer vehicles (0–10 years).
- A flattened curve for vehicles (20+ years) older, where additional aging shows diminished value.

This behavior closely matches real-world depreciation curves, confirming that the model captures the non-linear relationship between vehicle age and market value.

3 Part II

3.1 Dimensionality Reduction, Linear

For Dimensionality reduction we selected a random sample of data around a (1000) observations from the original dataset. This prevents crashes with large dataset.

```
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled_sample)

df_pca = X_pca.copy()
df_pca.columns = ['PC1', 'PC2']
df_pca['price'] = y_sample
```


We apply 2 components PCA, fit it on the sampled dataset and apply `y_sample` indices to the new dataset. The Price indices are binned in to five categories for easy visualized interpretation and plotted using a *snsscatterplot*.

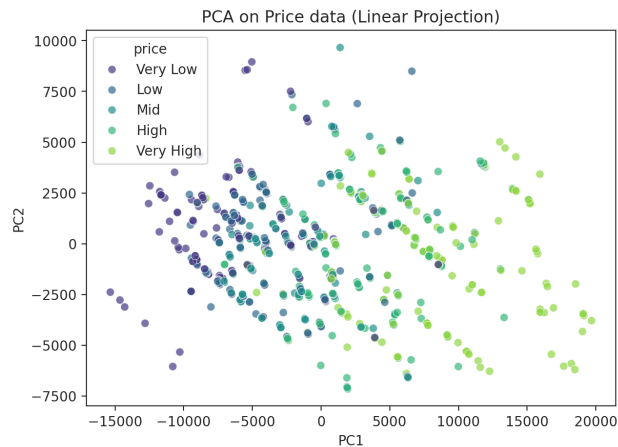


Figure 7: PCA Linear Projection

The PCA projection in Figure 7 shows a diagonal separation for the price categories. Very Low prices (dark purple) cluster toward the left region. while very High prices (bright green) cluster toward the right region. Mid-range prices transition between these extremes. PC1 seems to be the primary driver for price variations, while PC2 provides secondary differentiation.

3.2 Dimensionality Reduction, Non-Linear

3.2.1 Multidimensional Scaling (MDS:)

This type of Reduction shows a much stretched or dispersed version of PCA on `n_component = 2` setting. Some separation is observed, but still overlapping. We use the same sampled original dataset for visualization

```
mds = MDS(n_components=2, random_state=42)
X_mds = mds.fit_transform(X_scaled_sample)
```

Upon increasing `n_component` from 2 to 3, In Figure 9 we have a diagonal separation. Higher priced car(yellow) on the lower end, while Lower priced cars(purple) are on the upper section of the separation. while the mid tier cars mostly occupy the central and upper region.

3.2.2 ISomap:

For this type of plot we set the `n_neighbors` value between the range of 15 to 30 and set `n_component` value to two.

```
isomap = Isomap(n_components=2, n_neighbors=15)
X_iso = isomap.fit_transform(X_scaled_sample)
```

The scatter plot shows stronger curvature and emerging cluster-like regions with partial separation between price groups. Reducing $n_neighbors$ from 15 to 10 (Figure 8) produces a more circular structure, where lower-priced cars appear in the lower-left, higher-priced cars in the upper-right, and mid-tier cars occupy the remaining regions.

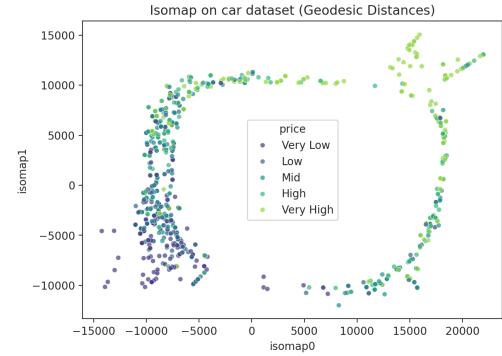


Figure 8: Isomap $n_neighbors = 10$

3.2.3 t-SNE:

With a $perplexity = 80$ & $n_components = 2$ we get a clear tight and well separated scatter plot. Price categories form distinct groups. The lower priced car occupy (left & upper right) side of the graph and converge to the lower right side of the graph, where the car prices are of the highest value.

```
tsne = TSNE(n_components=2, perplexity=80, random_state=42)
X_tsne = tsne.fit_transform(X_scaled_sample)
```

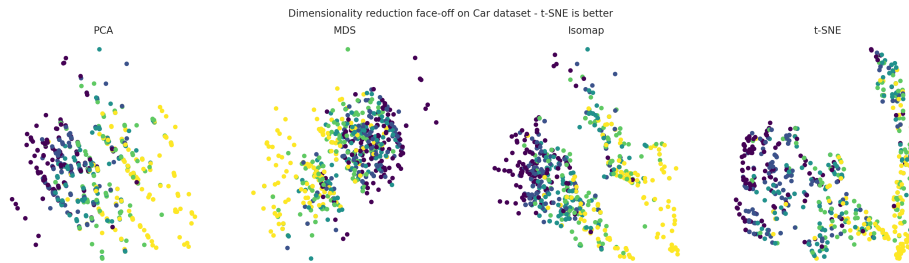


Figure 9: dimensionality reduction comparison

3.3 Polynomial Regression

Use the sampled dataset again for regression model focusing on *mileage_per_year* feature.

```
X_original = X_scaled_sample['mileage_per_year'].values
y_original = y_sample.values
```

Use manual log transformation for target rather than *TransformedTargetRegressor* and normalize *mileage_per_year* for stability due to large values.

```
y_log = np.log1p(y_original)
X_p = (X_original - X_original.mean()) / X_original.std()
models_poly = {}
```

Next, for polynomial feature generation we create columns $(X^1, X^2, X^3, \dots X^n)$.

```
X_poly_features = np.column_stack([X_p**i for i in range(1, degree+1)])
X_poly_sm = sm.add_constant(X_poly_features)

glm_poly = sm.GLM(y_log, X_poly_sm, family=sm.families.Gaussian(link=sm.families.links.Identity()))
results_poly = glm_poly.fit(dis=0)
```

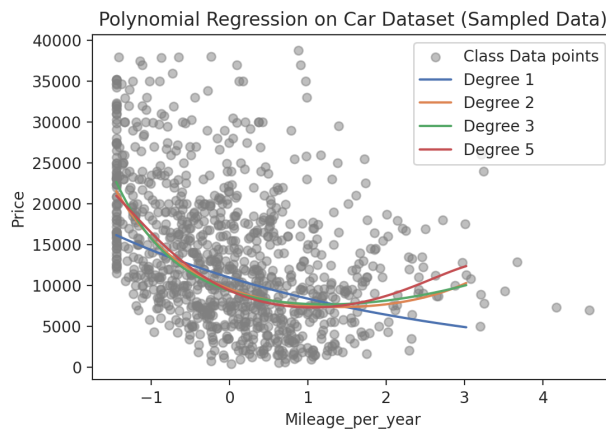


Figure 10: Polynomial Regression

```
X_plot = np.linspace(np.percentile(X_original, 1), np.percentile(X_original, 99), 200)
X_plot_scaled = (X_plot - X_original.mean()) / X_original.std()
```

Due to large *mileage_per_year* value Z-score normalization is applied before visualization of the data-points. Reason for this is due to polynomial features scale which rises exponentially (large X^2).

Based on the results from Figure 10 *degree* 2 and 3 gives the best fit. Captures, the essential non-linearity following the natural pattern of the dataset: rapid depreciation, then stabilization. while degree 1 and 5 show signs of under or overfitting.

3.4 Clustering for Feature Engineering

3.4.1 K-Mean:

For this specific analysis two unsupervised clustering approaches are applied. For K-mean identifies 3 well separated clusters, while DBSCAN detects density based clusters with significant noise.

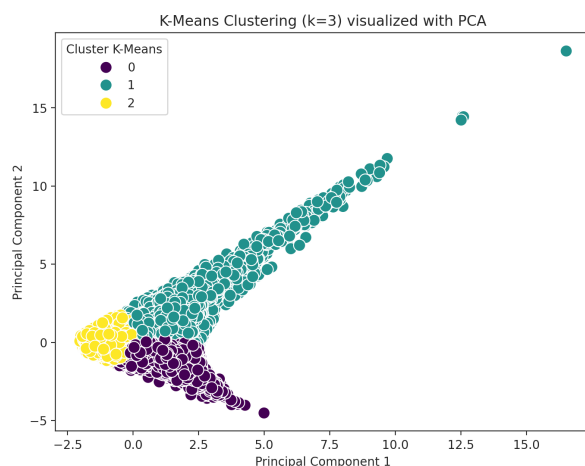


Figure 11: K-Mean Clustering (k=3)

Apply Gaussian for continuous target and fit the polynomial model. With multiple **degree** values ranging from 1 to 5 we can plot and find the degree model based on the result.

Apply and use percentile method for outlier removal, between 1st to 99th percentile with 200 equally spaced points for smooth curves.

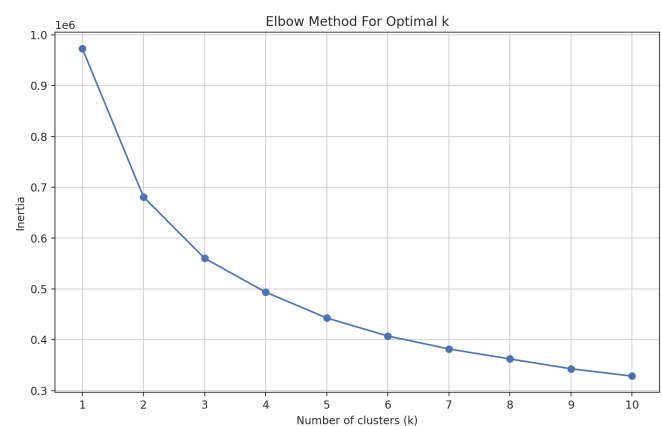


Figure 12: Optimal K using Elbow Method

As clustering is performed on high-dimensional data, visualization can become tricky. Hence a dimensionality reduction method like PCA can be used to project the data on a 2D plane. For this purpose we use PCA method for visualization of the *KMean* data points. To find the optimal K, Tests from k=1 to k=10 for comprehensive evaluation . Using a *for* loop we apply *KMeans* for all the values ranging from 1 to 10. Inertia plot in Figure 12 shows a decreasing inertial graph as the number of clusters increase.

```
inertia = []
K = range(1, 11)
for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled_df)
    print(f"fitted K{k}")
    inertia.append(kmeans.inertia_)
```

Knee Locator: To find the optimal *K* value a *KneeLocator* library is used by providing the array of inertia values we get *optimal_k* for our model.

```
kl = KneeLocator(K, inertia, curve='convex', direction='decreasing')
optimal_k = kl.elbow
```

3.5 DBSCAN:

The Clustering model was applied with parameters as (*eps* = 3 & *min_samples* = 6). It identifies mostly one large cluster with scattered micro-clusters. Mostly identifies data points as -1 outliers in the dataset.

```
dbscan = DBSCAN(eps=3, min_samples=6)
labels_dbscan = dbscan.fit_predict(X_scaled_sample)
```

Since similar visualization methods where used this suggests that PCA might not be the optimal method to identify density based clusters in a large data-set.